

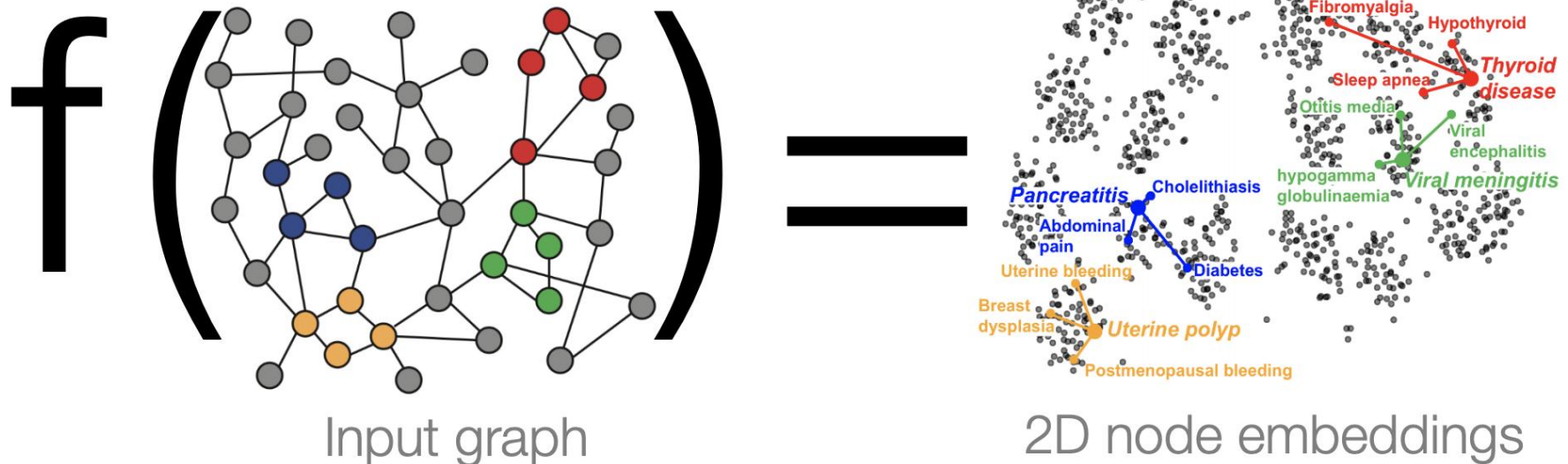
# ECE 4502/6502 & CS 6501: Machine Learning on Graphs

## Lecture 12. Graph Neural Networks I

Instructor: Jundong Li  
Spring 2025, University of Virginia

# Node Embeddings

- **Intuition:** Map nodes to  $d$ -dimensional embeddings such that similar nodes in the graph are embedded close together

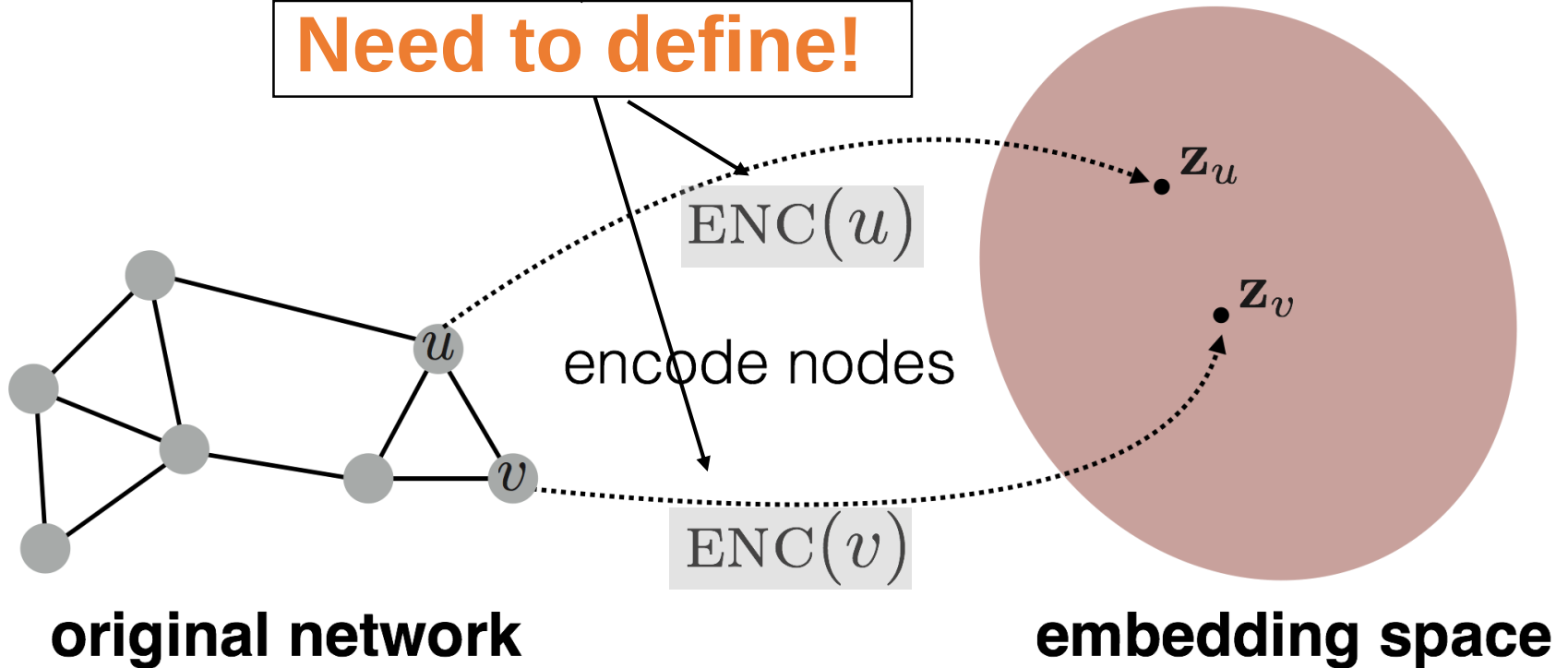


How to learn the mapping function  $f$ ?

# Node Embeddings

**Goal:**  $\text{similarity}(u, v) \approx \mathbf{z}_v^\top \mathbf{z}_u$

**Need to define!**



# Two Key Components

- **Encoder** maps each node to a low-dimensional vector

$$\text{ENC}(v) = \mathbf{z}_v$$

node in the input graph

d-dimensional embedding

- **Similarity function** specifies how relations in vector space map to relations in the original network

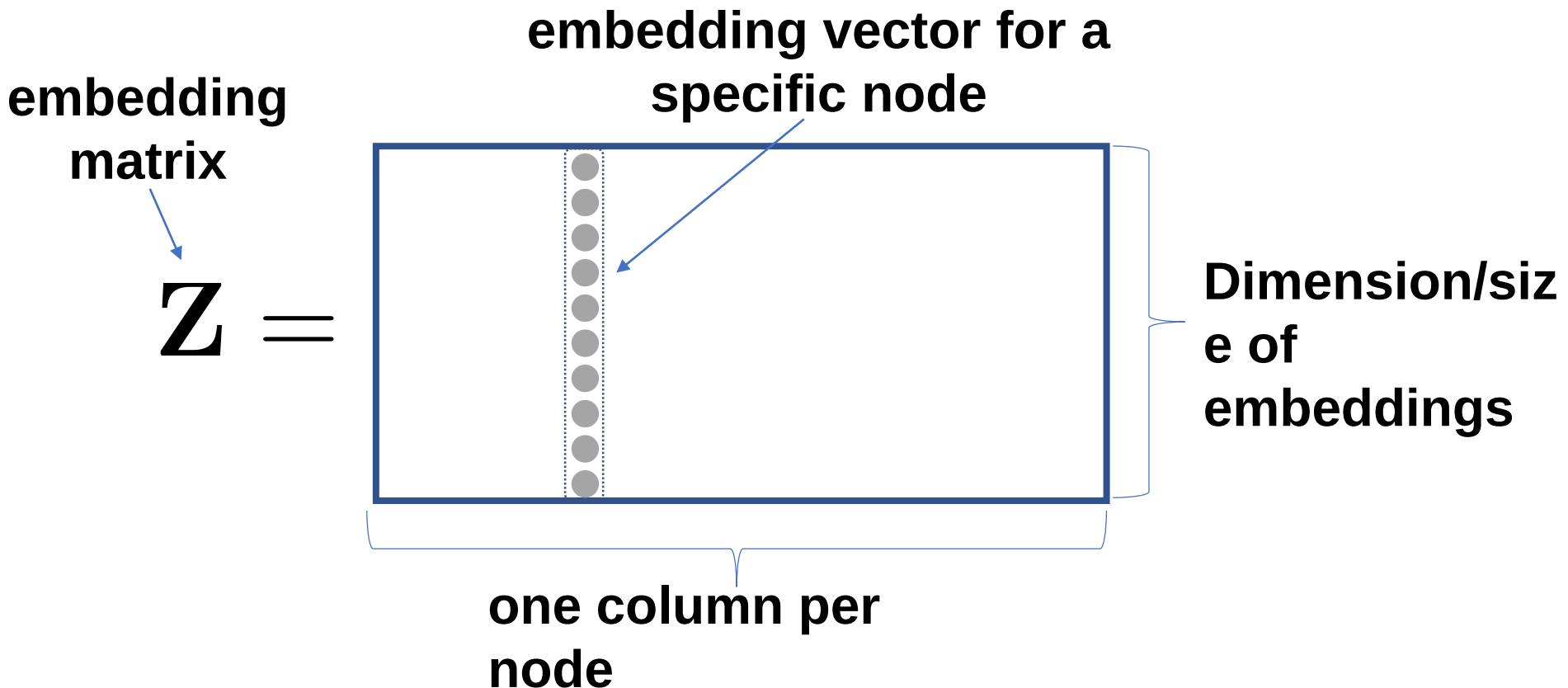
$$\text{similarity}(u, v) \approx \mathbf{z}_v^\top \mathbf{z}_u$$

Similarity of  $u$  and  $v$  in the original network

dot product between node embeddings

# From “Shallow” to “Deep”

- So far we have focused on “shallow” encoders, i.e., **embedding lookups**



# Shallow Encoders

- Limitations of shallow embedding methods:
  - $O(|V|)$  parameters are needed
    - No sharing of parameters between nodes
    - Every node has its own unique embedding
  - Inherently “transductive”
    - Cannot generate embeddings for nodes that are not seen during training
  - Do not incorporate node features
    - Many graphs have features that we can and should leverage

# From “Shallow” to “Deep”

- We will now discuss “deeper” methods based on **graph neural networks**

$$\text{ENC}(v) = \text{multiple layers of non-linear transformations of graph structure}$$

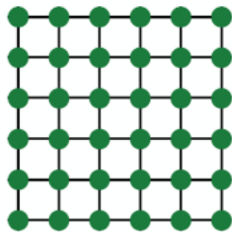
- In general, all of these more complex encoders can be combined with the similarity functions from our last class

# Deep Learning on Image and Text

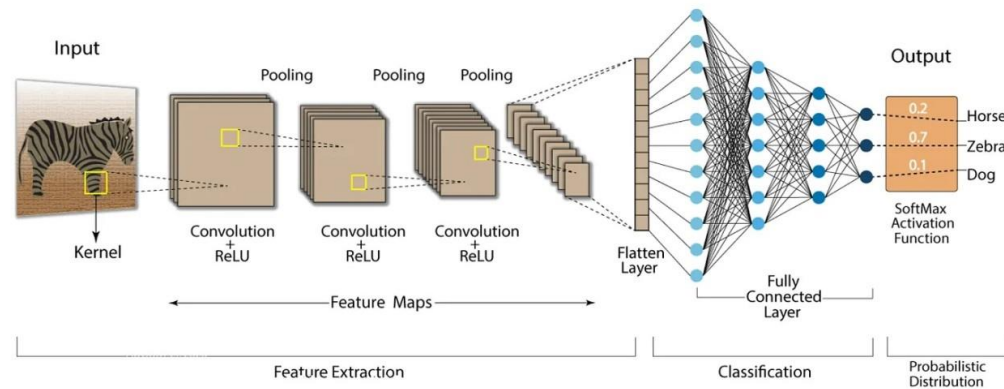
- Modern machine learning tools are predominantly designed for **grid** or **sequence** data

## Computer Vision

### Grid Data



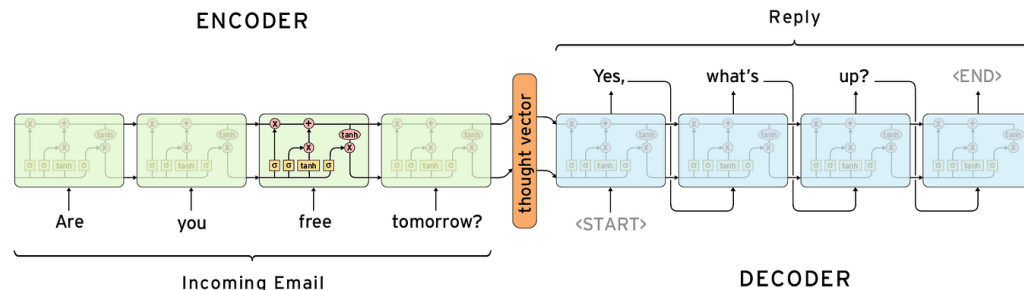
Convolution Neural Network (CNN)



## Natural Language Processing

### Sequence Data

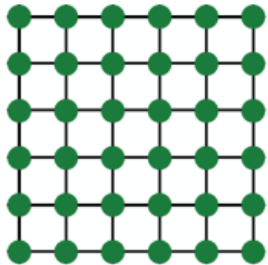
### Sequence Data





# Transfer to Graph Data

- Graphs have complex topological structure

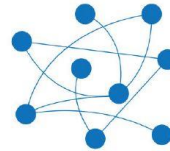


**Images**

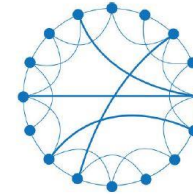


**Text**

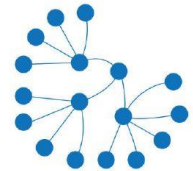
**VS.**



**Random**  
Average distributions.  
No structure or hierarchical patterns.



**Small-World**  
High local clustering and short  
average path lengths.  
Hub-and-spoke architecture.



**Scale-Free**  
Hub-and-spoke architecture preserved at  
multiple scales.  
High power law distribution.

**Networks**

- The nodes and edges can have complex features
- No fixed node ordering or reference point
- The graph can evolve over time

# Outline

- Basics of deep learning for graphs
- Graph convolutional networks

# **Basics of Deep Learning for Graphs**

# Basics of Deep Learning

- Loss function  $\mathcal{L}$ :

$$\min_{\Theta} \mathcal{L}(\mathbf{y}, f_{\Theta}(\mathbf{x}))$$

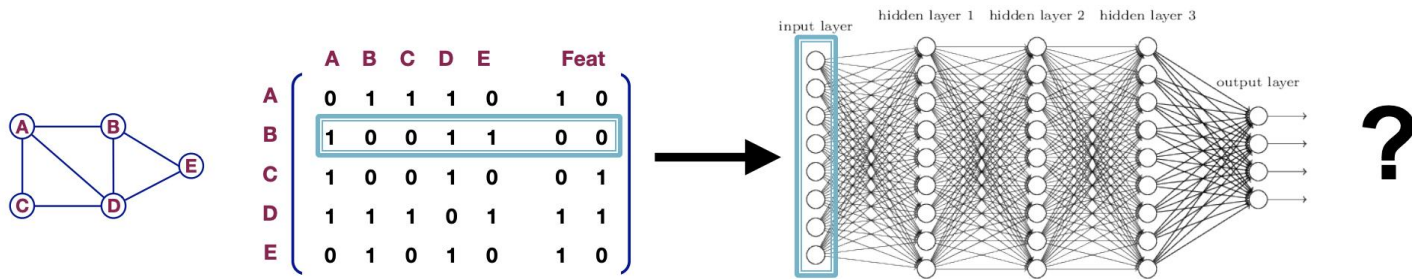
- $f$  can be a simple linear layer, an MLP, or other neural networks (e.g., a GNN later)
- Sample a minibatch of input data  $\mathbf{x}$
- **Forward propagation:** Compute  $\mathcal{L}$  given  $\mathbf{x}$
- **Back-propagation:** Obtain gradient  $\nabla_{\Theta} \mathcal{L}$  using a chain rule.
- Use stochastic gradient descent (SGD) to optimize  $\mathcal{L}$  for weights  $\Theta$  over many iterations.

# Problem Setup for Graphs

- Assume we have a graph  $G$ :
  - $V$  is the vertex set.
  - $A$  is the adjacency matrix (assume binary)
  - $X \in \mathbb{R}^{m \times |V|}$  is a matrix of node features
    - Social networks: user profile, user image
    - Biological networks: gene expression profiles, gene functional information
    - No features:
      - Indicator vector (one-hot encoding of a node)
      - Vector of constant 1:  $[1, 1, \dots, 1]$

# A Naïve Approach

- Given a graph  $G(A, X)$ 
  - Join adjacency matrix  $A$  and features  $X$
  - Use them as input for deep neural network



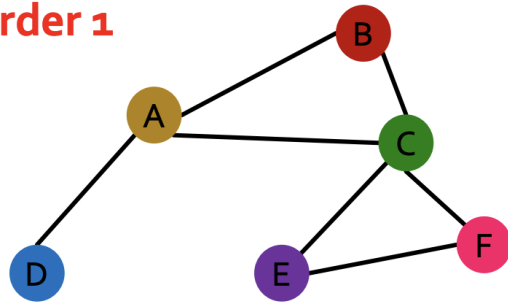
- Issues with this idea
  - $O(|V|)$  parameters
  - Cannot easily adapt to graphs with different sizes
  - Sensitive to node ordering

# Permutation Invariance

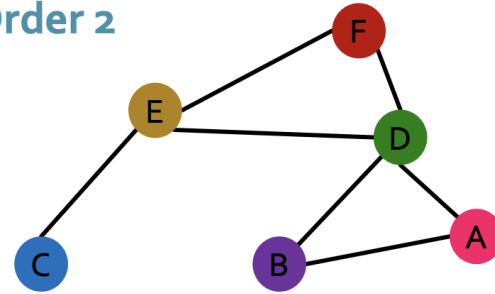
- Consider we want to embed an entire graph
- **Observation:** A graph does not have a canonical ordering of its nodes
  - We can have many different node orderings of the same

graph

Order 1



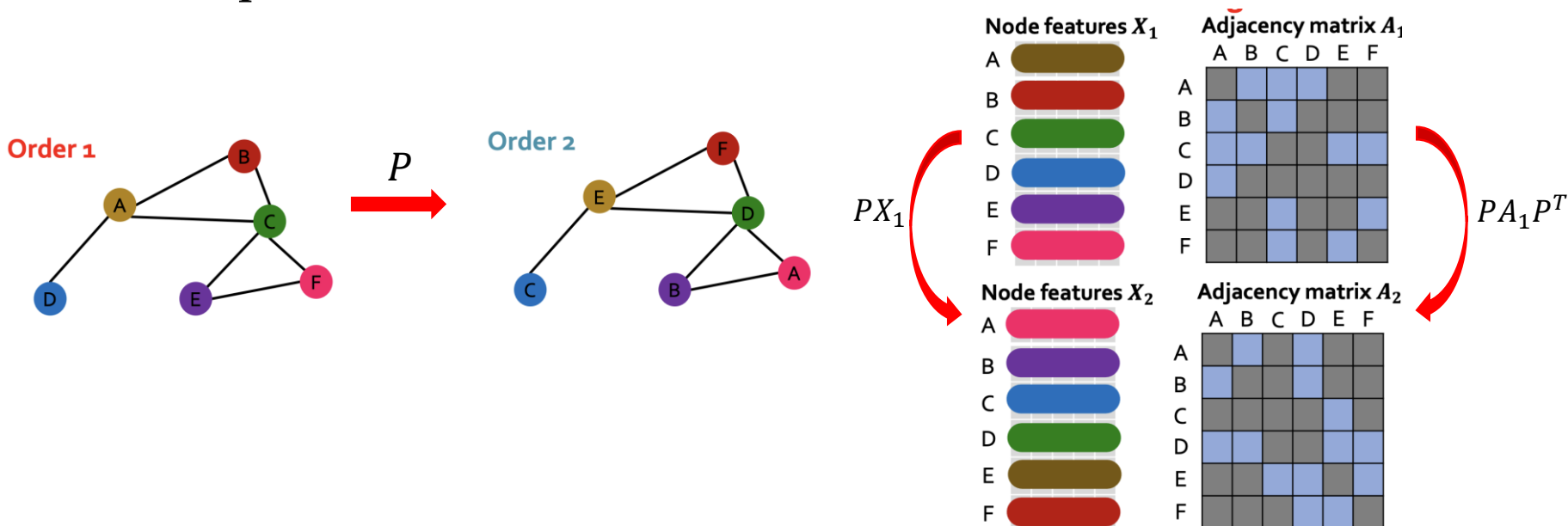
Order 2



- **Desired characteristics:** the learnt embedding function should give the same embedding result regardless of how the nodes are numbered.

# Permutation Invariance

- Definition: For any graph function  $f: \mathbb{R}^{|V| \times |V|} \times \mathbb{R}^{|V| \times m} \rightarrow \mathbb{R}^d$ ,  $f$  is permutation invariant if  $f(A, X) = f(PAP^T, PX)$  for any permutation  $P$ .
  - $m$ : dimension of node feature
  - $d$ : dimension of embedding vector of  $G(A, X)$
  - $P$ : permutation matrix that shuffle the node order

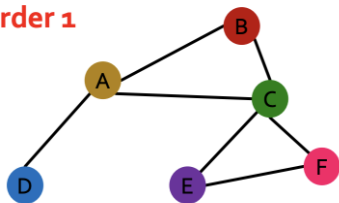




# Permutation Equivariance

- Consider we want to embed the nodes
- **Desired characteristics:** the learnt node embeddings should be the same regardless of how the nodes are numbered.

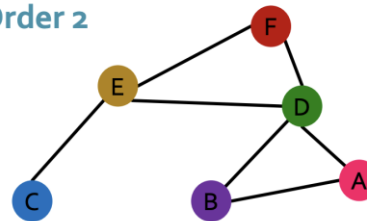
Order 1



$$f(A_1, X_1) =$$

|   |  |  |
|---|--|--|
| A |  |  |
| B |  |  |
| C |  |  |
| D |  |  |
| E |  |  |
| F |  |  |

Order 2



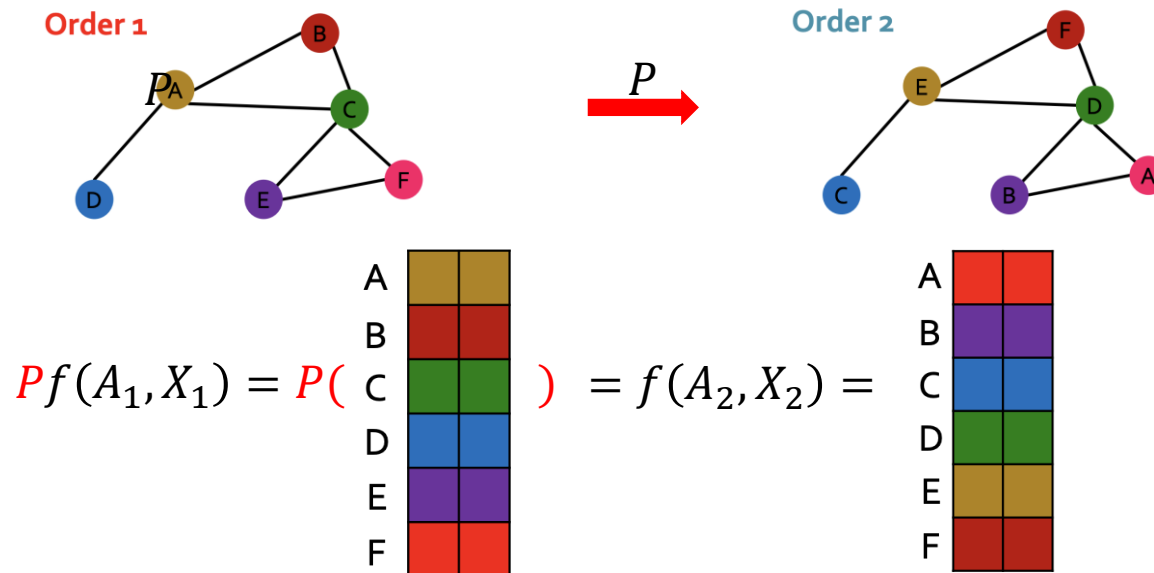
$$f(A_2, X_2) =$$

|   |  |  |
|---|--|--|
| A |  |  |
| B |  |  |
| C |  |  |
| D |  |  |
| E |  |  |
| F |  |  |

For two orderings, the embedding vector for the node at the same position should be the same!

# Permutation Equivariance

- Definition: For any node function  $f: \mathbb{R}^{|V| \times |V|} \times \mathbb{R}^{|V| \times m} \rightarrow \mathbb{R}^{|V| \times d}$ ,  $f$  is permutation equivariant if  $Pf(A, X) = f(PAP^T, PX)$  for any permutation  $P$ .
  - $m$ : dimension of node feature
  - $d$ : dimension of embedding vector of  $G(A, X)$
  - $P$ : permutation matrix that shuffle the node order



# Invariance and Equivariance

- Permutation invariant (graph-level functions)

$$f(A, X) = f(PAP^T, PX)$$

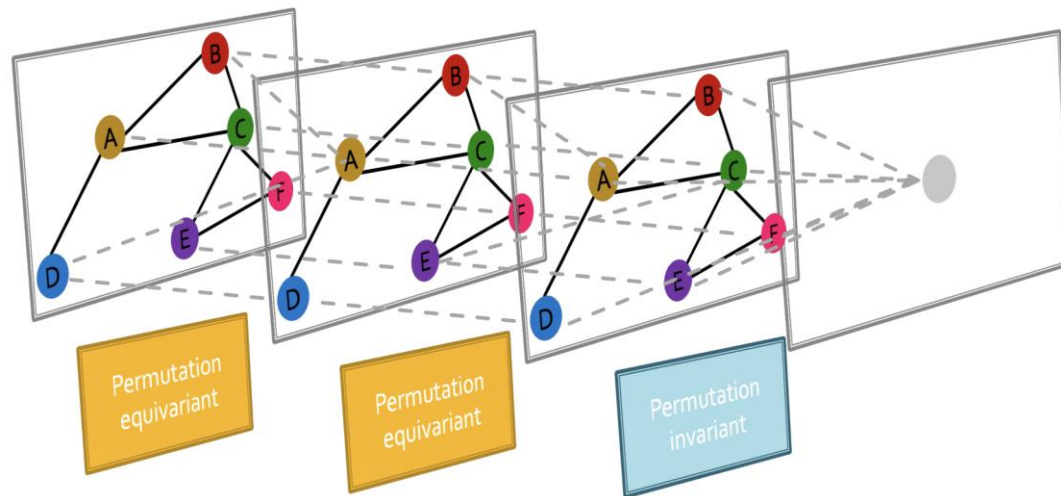
- Permutation equivariant (node-level functions)

- Examples  $Pf(A, X) = f(PAP^T, PX)$

- $f(A, X) = 1^T X$ : permutation invariant (element-wise feature summation)
  - $f(PAP^T, PX) = 1^T PX = 1^T X = f(A, X)$
- $f(A, X) = X$ : permutation equivariant (node features)
  - $f(PAP^T, PX) = PX = Pf(A, X)$
- $f(A, X) = AX$ : permutation equivariant (neighborhood feature aggregation)
  - $f(PAP^T, PX) = PAP^T PX = PAX = Pf(A, X)$

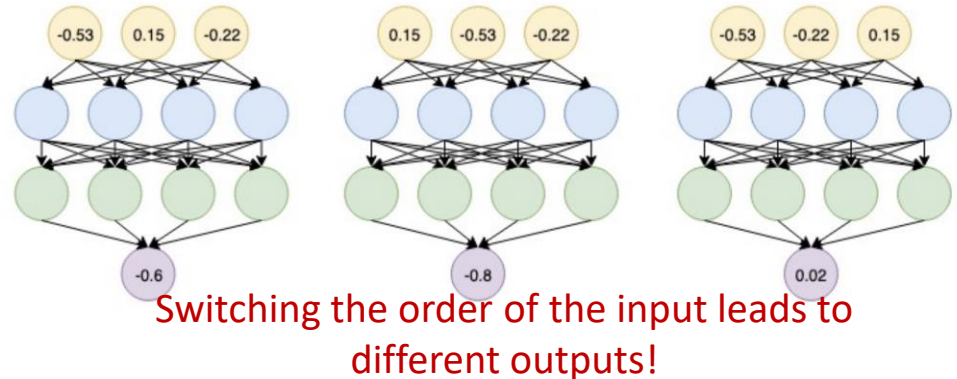
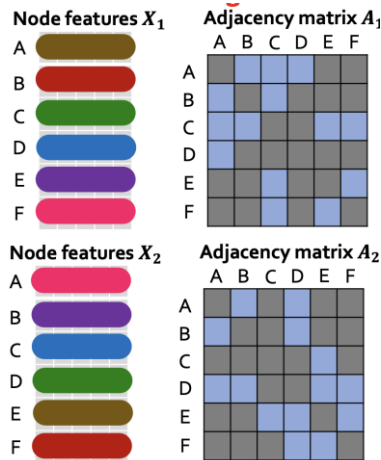
# Graph Neural Network Overview

- Graph neural networks (GNN) consist of multiple permutation equivariant/invariant functions

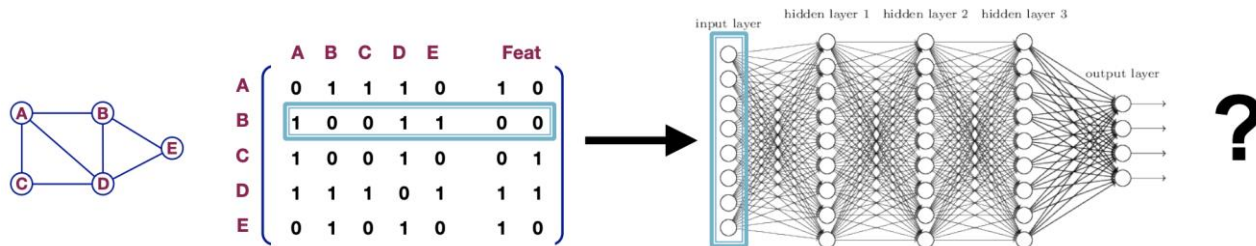


# The Naïve Approach

- The naïve approach is not permutation invariant/equivariant
  - MLP is not permutation invariant/equivariant



- Permutated graph leads to permutated MLP input

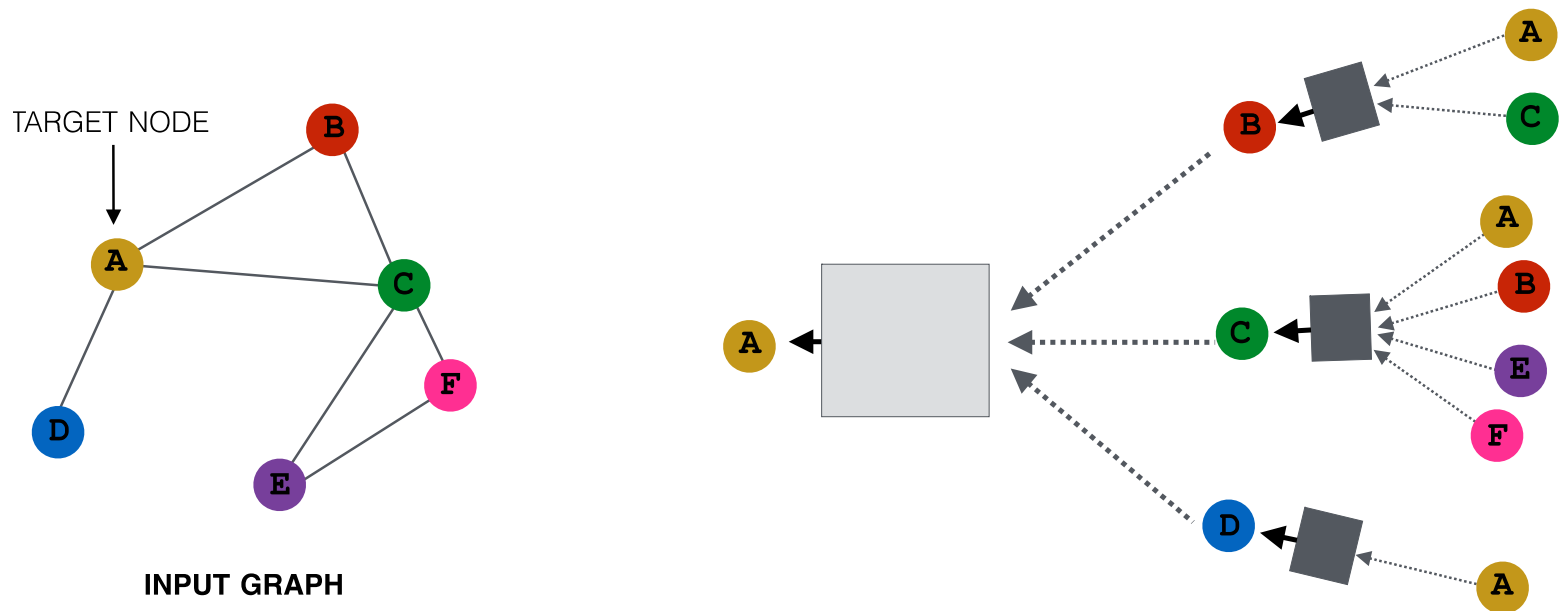


# Graph Convolutional Networks

- Based on material from: Kipf et al., 2017. [Semisupervised Classification with Graph Convolutional Networks](#). *ICLR*

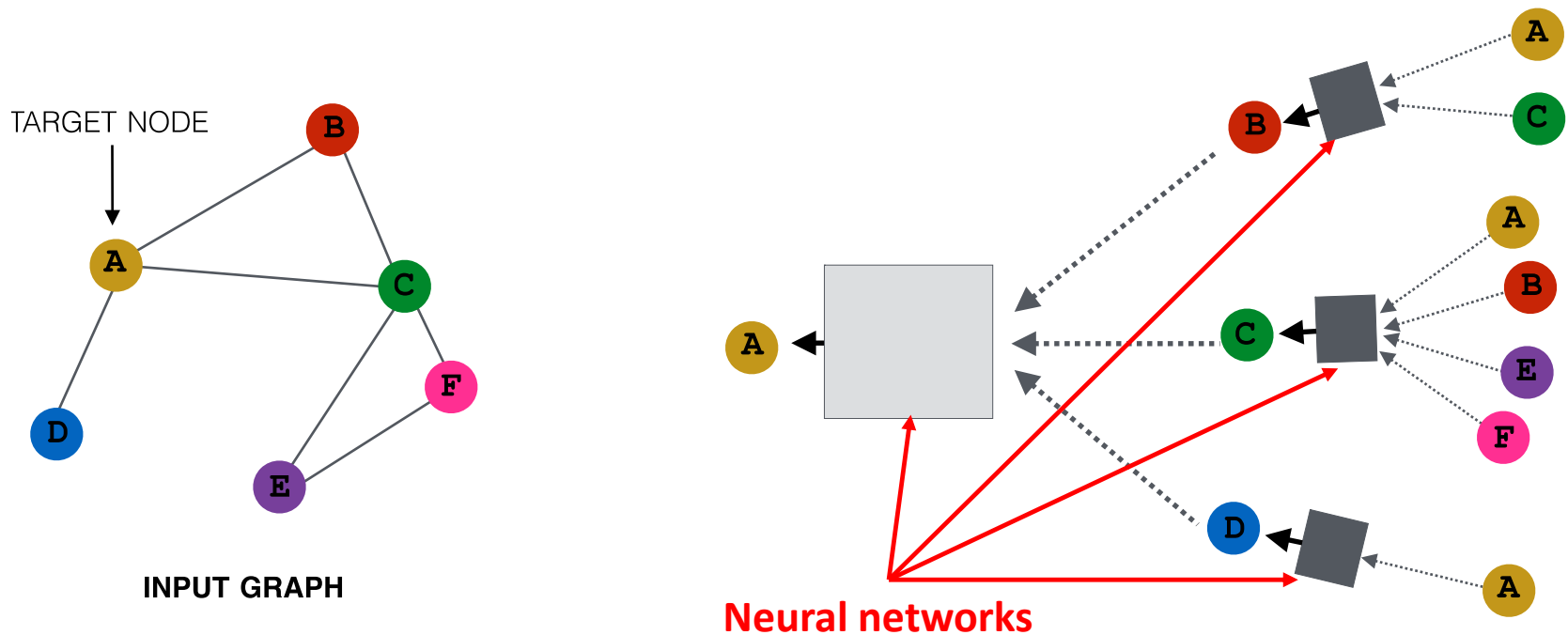
# Neighborhood Aggregation

- **Key idea:** Generate node embeddings based on local neighborhoods



# Neighborhood Aggregation

- **Intuition:** Nodes aggregate information from their neighbors using neural networks

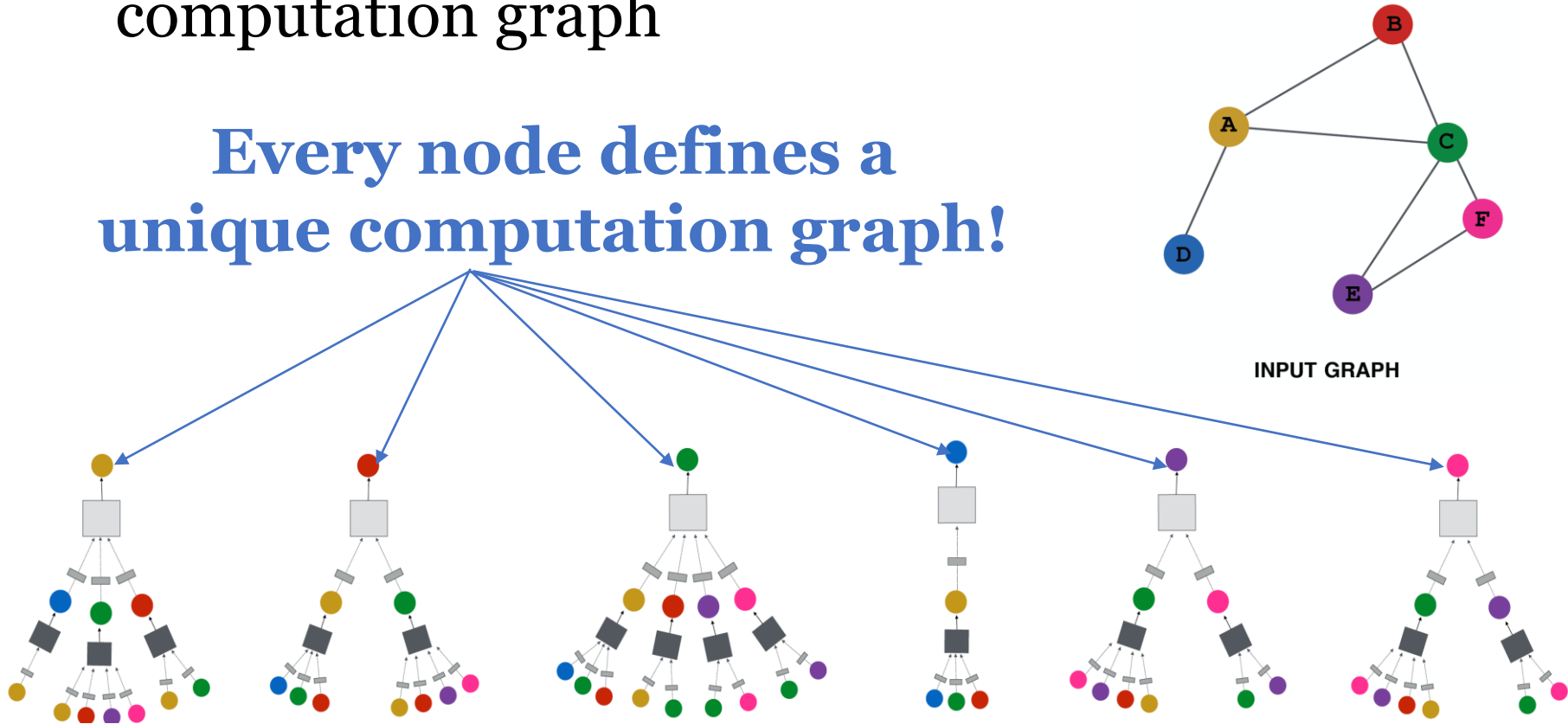




# Neighborhood Aggregation

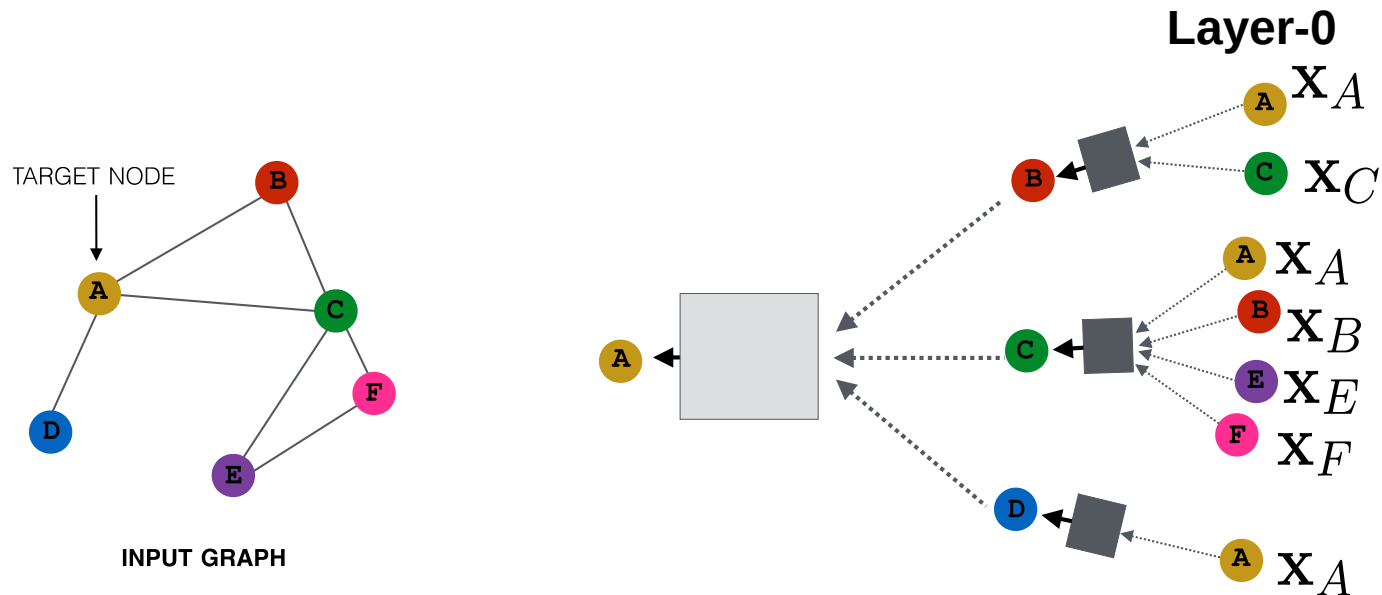
- **Intuition:** Network neighborhood defines a computation graph

Every node defines a unique computation graph!



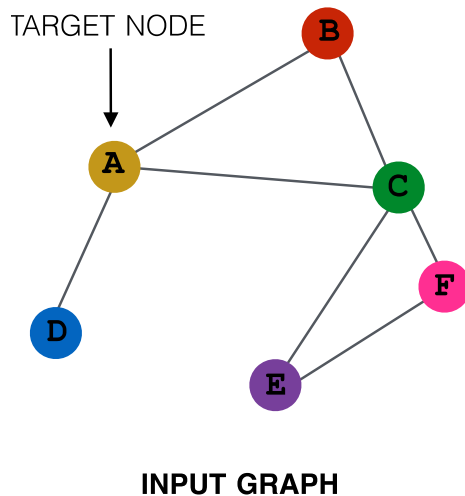
# Deep Model: Many Layers

- Model can be of arbitrary depth:
  - Nodes have embeddings at each layer
  - Layer-0 embedding of node  $u$  is its input feature,  $x_u$
  - Layer-K embedding gets information from nodes that are K hops away

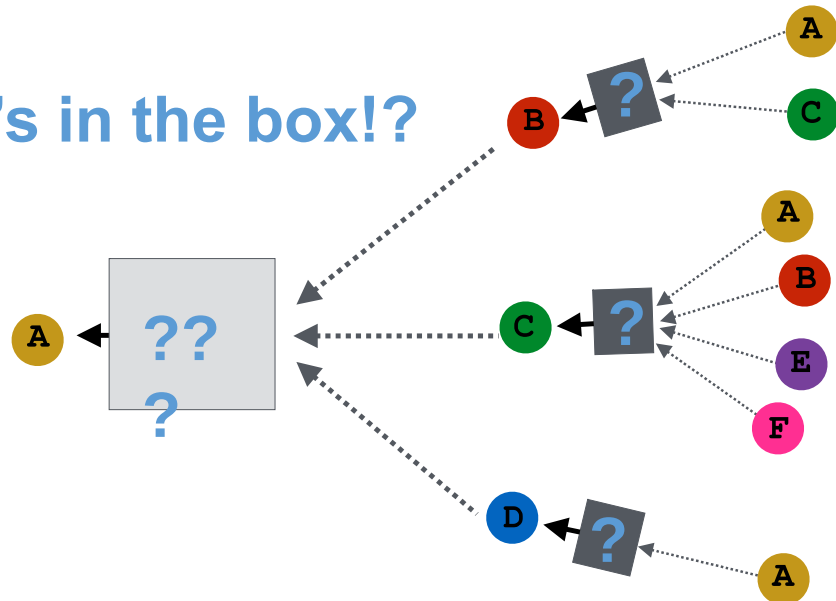


# Neighborhood Aggregation

- **Neighborhood Aggregation:** key distinctions are in how different approaches aggregate information across the layers

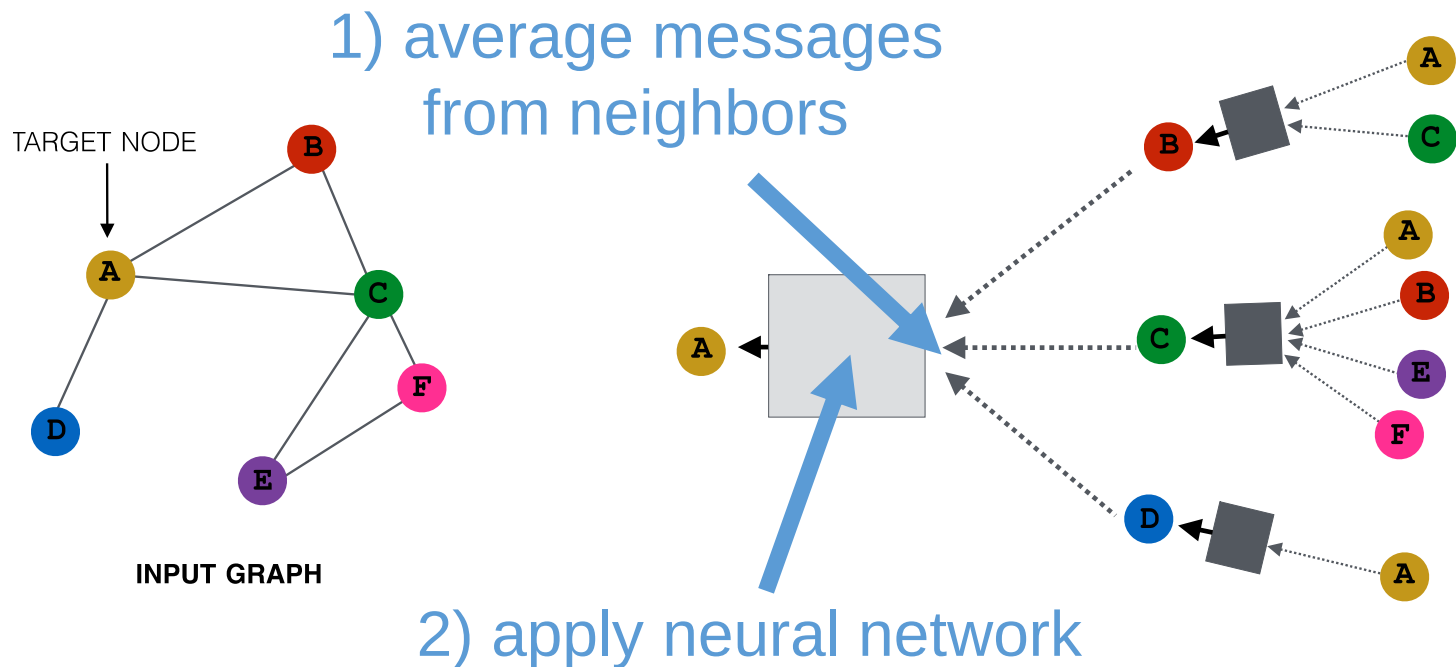


what's in the box!?



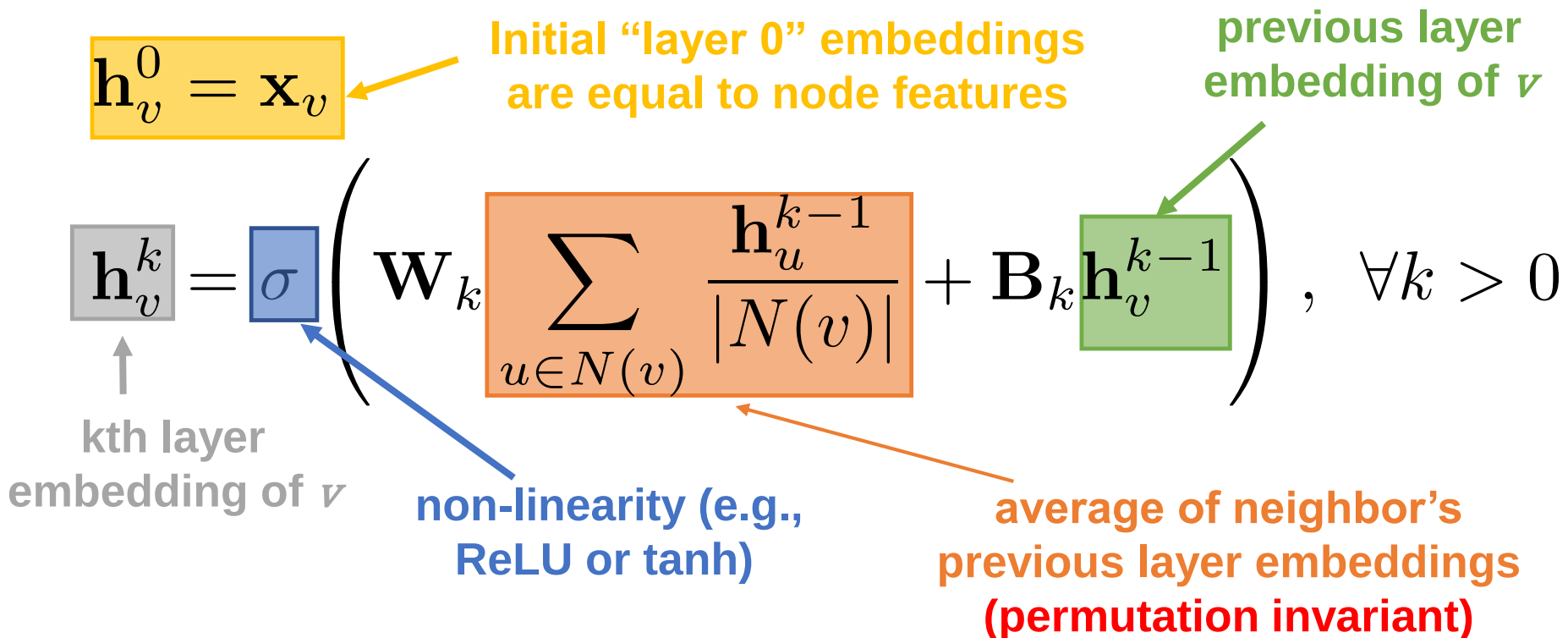
# Neighborhood Aggregation

- **Basic approach:** Average neighbor information and apply a neural network



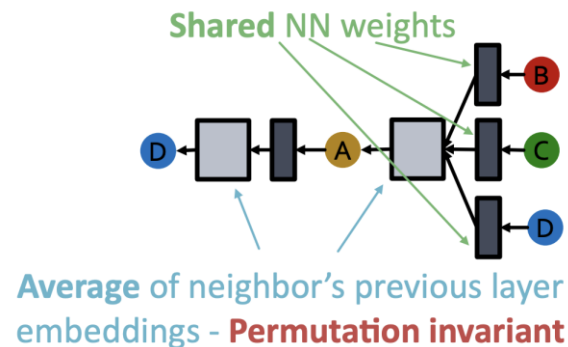
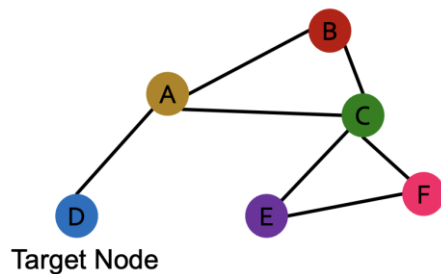
# The Math: Deep Encoder of GCN

- **Basic approach:** Average neighbor messages and apply a neural network.



# GCN: Invariance and Equivariance

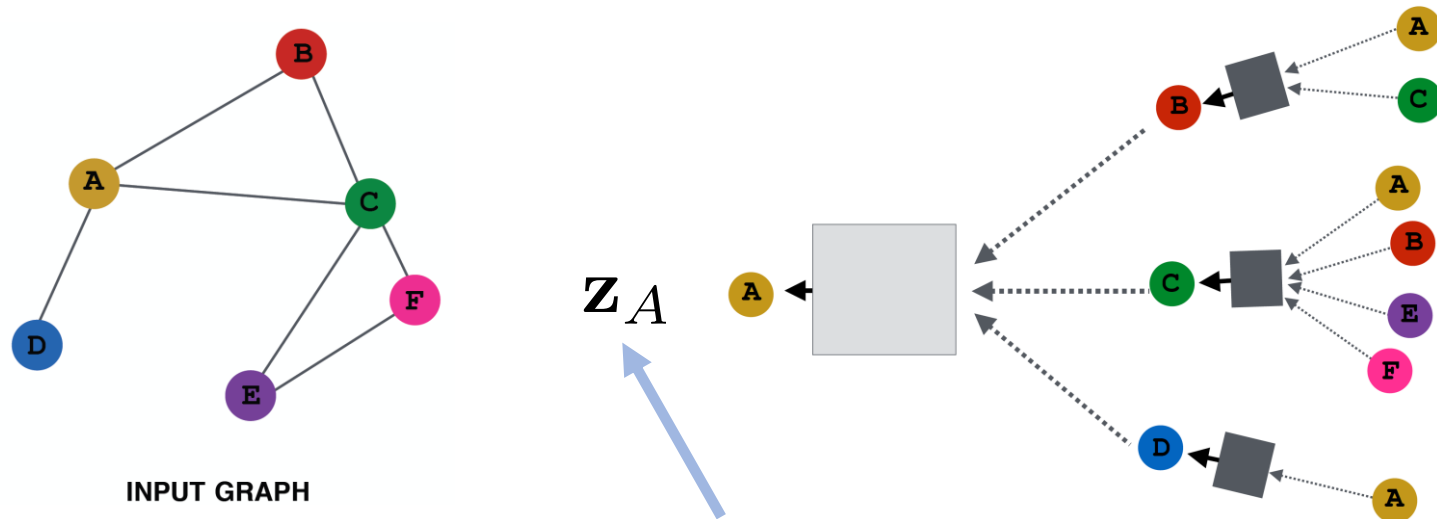
- The embeddings generated by GCN is permutation equivariant
  - The embedding of a given node under GCN is permutation invariant



- The embeddings for all the nodes in the graph under GCN is therefore permutation equivariant

# Training the Model

- How do we train the model to generate “high-quality” embeddings?



**Need to define a loss function  
on the embeddings,  $L(z_u)$ !**

# Model Parameters

$\mathbf{h}_v^0 = \mathbf{x}_v$

trainable matrices (i.e.,  
what we learn)

$$\mathbf{h}_v^k = \sigma \left( \mathbf{W}_k \sum_{u \in N(v)} \frac{\mathbf{h}_u^{k-1}}{|N(v)|} + \mathbf{B}_k \mathbf{h}_v^{k-1} \right), \quad \forall k \in \{1, \dots, K\}$$

$$\mathbf{z}_v = \mathbf{h}_v^K$$

- After K-layers of neighborhood aggregation, we get output embeddings for each node
- **We can feed these embeddings into any loss function** and run stochastic gradient descent to train the aggregation parameters





**Questions?**